

COMPUTER NETWORKS 2024/2025

class 0101 - Stop&Wait and Sliding Window
for reliable transmission

17/03/2025

Pedro Patinho <pp@di.uevora.pt>

(Based on slides by Prof. José Legatheaux Martins)



Objectives

- The Internet does not guarantee the delivery of packets as they may be lost or delivered in a disorderly manner.
- It then becomes necessary to find a way to transfer data reliably between two computers.
- In this class we will see the simplest protocol that exists to accomplish this – the *stop & wait* protocol

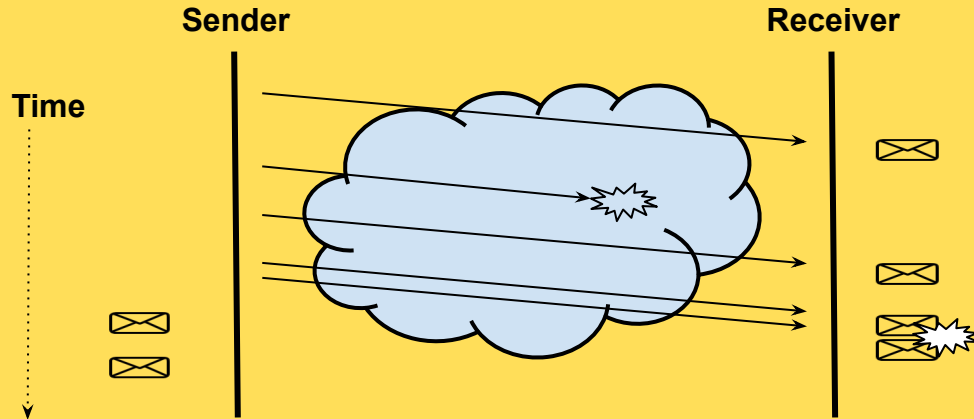
Hypothesis and Objective

- In channels, frames with errors are rejected by error control mechanisms and thus errors are transformed into losses (omissions)
- The network can also introduce packet loss or change the packet delivery order.
- The network may lose packets, but it doesn't lose them all, sooner or later some packets will reach the receiver
- Objective: You want to transfer data reliably and maximize throughput, or throughput, end-to-end

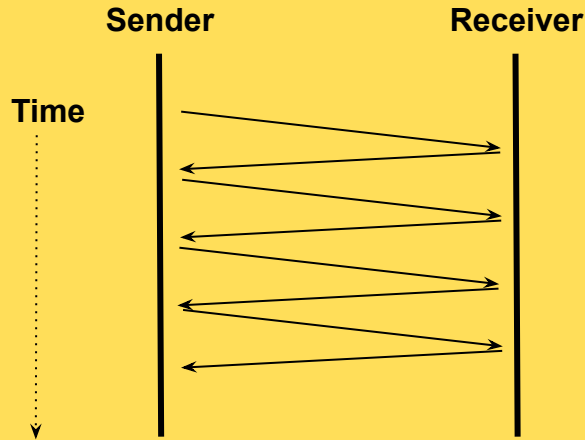
Problems

End-to-end error control and flow control is required.

Error control should mask packet loss and out-of-order arrivals. Flow control should prevent a very fast sender from “drowning” a slow receiver.

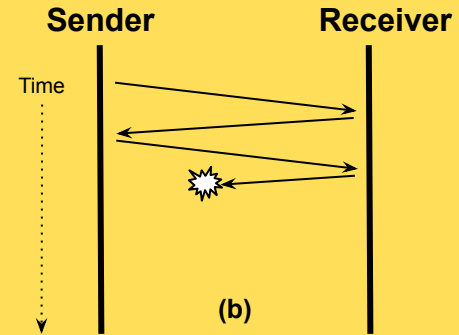
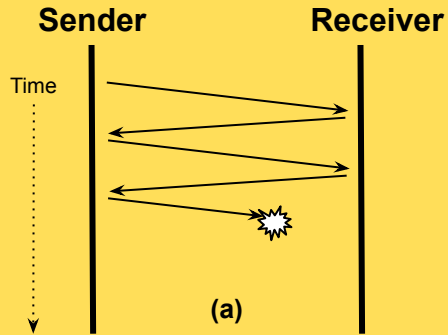


Receipt Acknowledgments (ACKs)

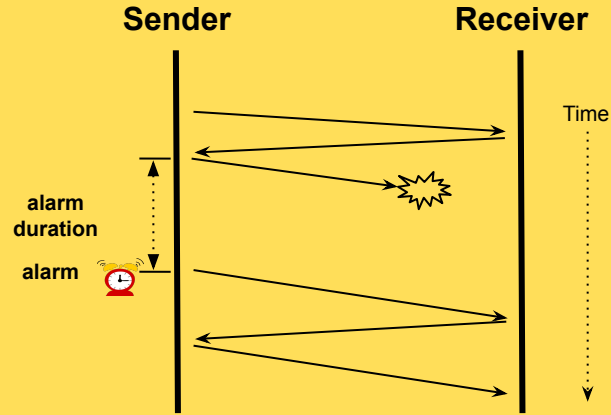


ACK = Acknowledgment (confirmation or acknowledgment of receipt
= you can continue)

Packet Loss

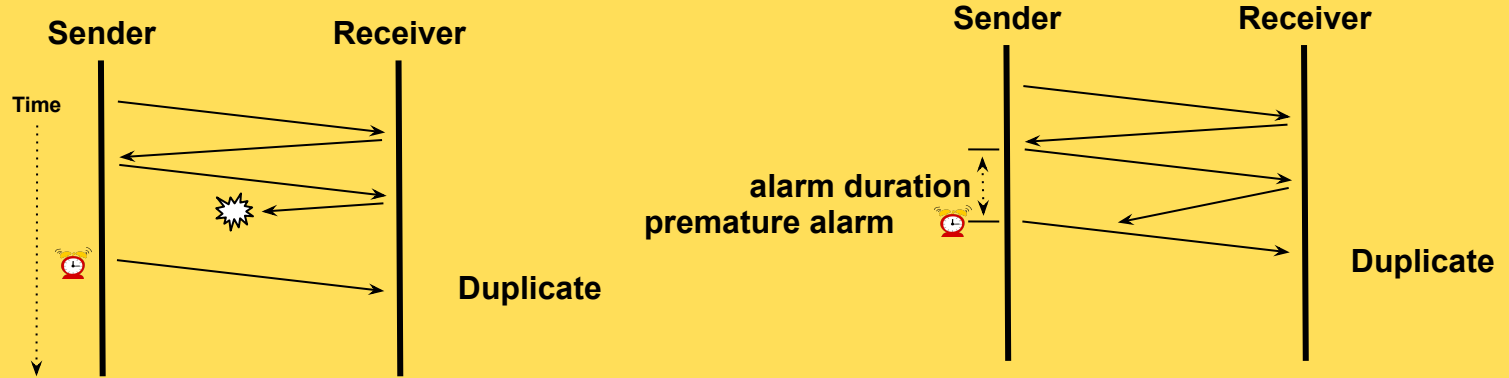


Timed Alarms

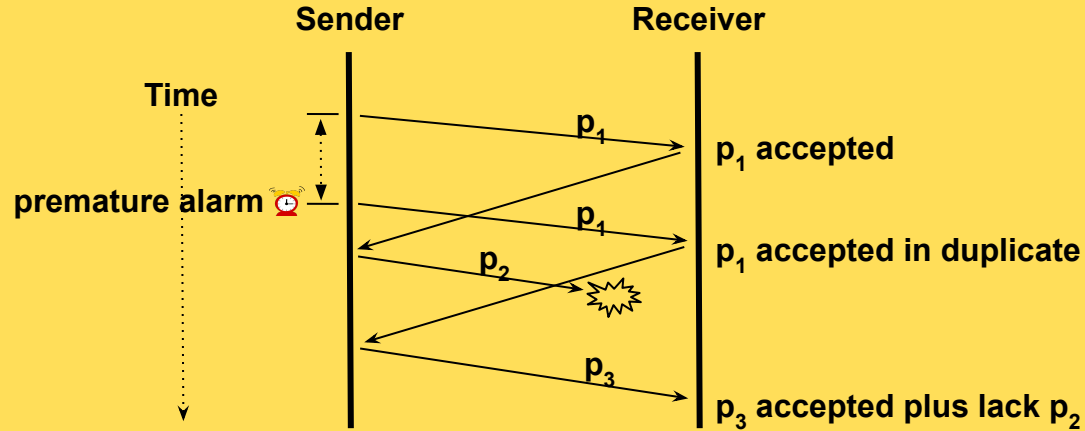


The terminology used in English in computer networks and distributed systems corresponds to the moment when the alarm timer ends: *timeout*

Anomalies and Alarms



Conjugated Anomalies

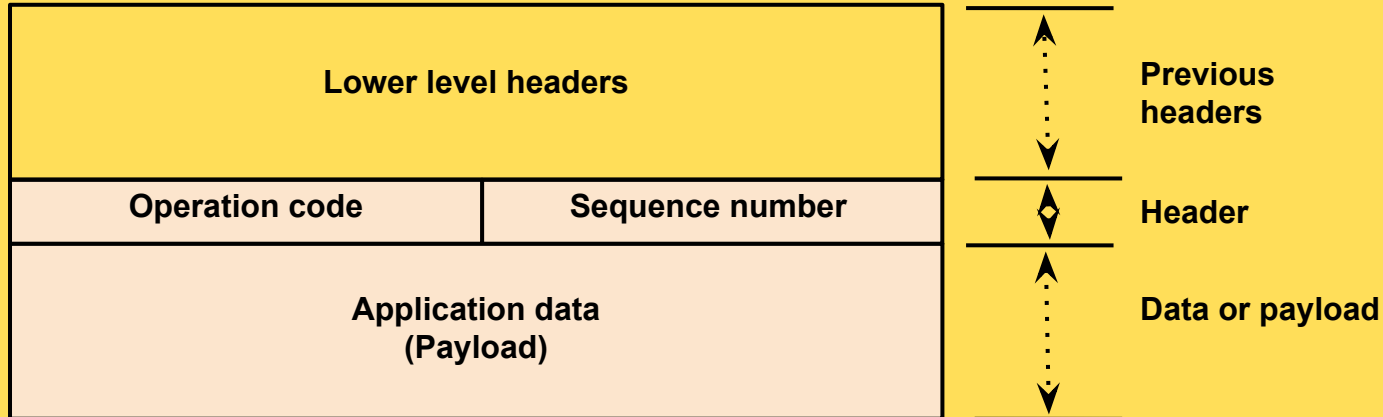


Problems still unresolved

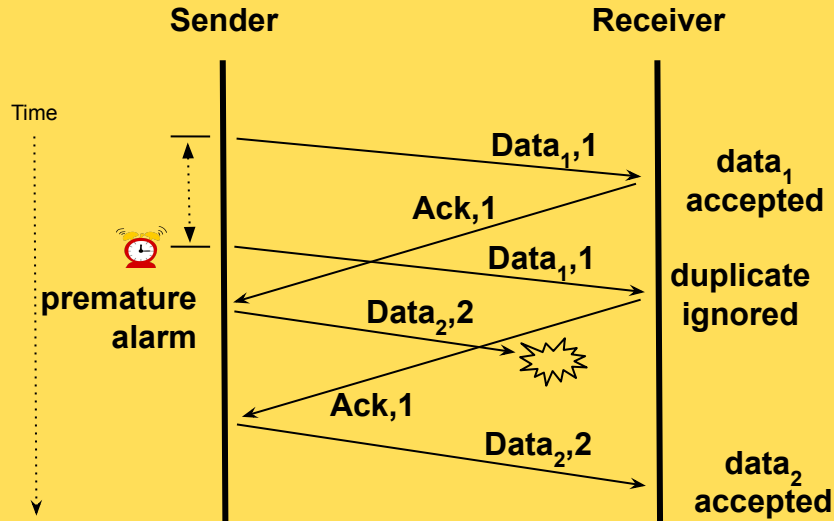
- The previous solution does not yet solve the problem of a missing ACK, which can lead to the acceptance of a duplicate packet.
- The same problem can be introduced by a slow receiver (originating a timeout) sending the ACK
- A badly adjusted alarm (timeout), too short for example, could lead to the same problem
- In fact, the relative speeds of the sender and receiver may not be constant, nor known a priori.
- Note that a too long alarm duration, a simplistic solution, leads to a overly slow recovery from a loss

Sequence numbers

- One solution is to introduce unique sequence numbers for each packet. These numbers allow the receiver to distinguish expected data, repeated data, etc.
- Traditionally, to save space in headers, it is important to minimize the number of bits to be used. The sequence number can then be reused cyclically as long as no confusion is introduced if the order of the messages is changed.



Stop & Wait Protocol



Operation

Sender:

- For each packet to be transmitted, give it a new sequence number, transmit it and activate an alarm. After:
 - If ACK arrives with expected sequence number – pass on
 - If the timer goes off – resend the packet and activate the alarm again
 - If ACK with wrong sequence number arrives – ignore it

Receiver:

- Whenever we receive a packet:
 - Send an ACK of the last correctly received packet. If the packet is new, keep it, otherwise ignore it.

Correction (*Safety*)

- Assuming that the network sooner or later manages to get a message delivered to the receiver, that is, that the network always allows some progress, *however small*:
- The protocol guarantees (try to demonstrate):
 - That each packet is correctly delivered to the receiver
 - That this packet is not confused with another
 - That all packets will reach the receiver

Alarm Regulation

- The alarm timer value (timeout) is important for error recovery
 - A value that is too large means that the sender is slow to recover from errors
 - An overly short value leads to useless duplicates
 - In any case, correction is not at issue as long as an alarm is always used and the receiver systematically sends an ACK for all packets received, bad or good.
- To avoid useless repetitions, the value can be relaxed
 - It is possible to try to estimate a more suitable value
 - Think how

Error-free performance

- Two computers A and B connected by a direct channel with a length of 10,000 km and a speed of 1 Mbps
- Packets with 1000 bytes
- Header bits are ignored
- Ignore the transmission time of an ACK
- Processing time is ignored

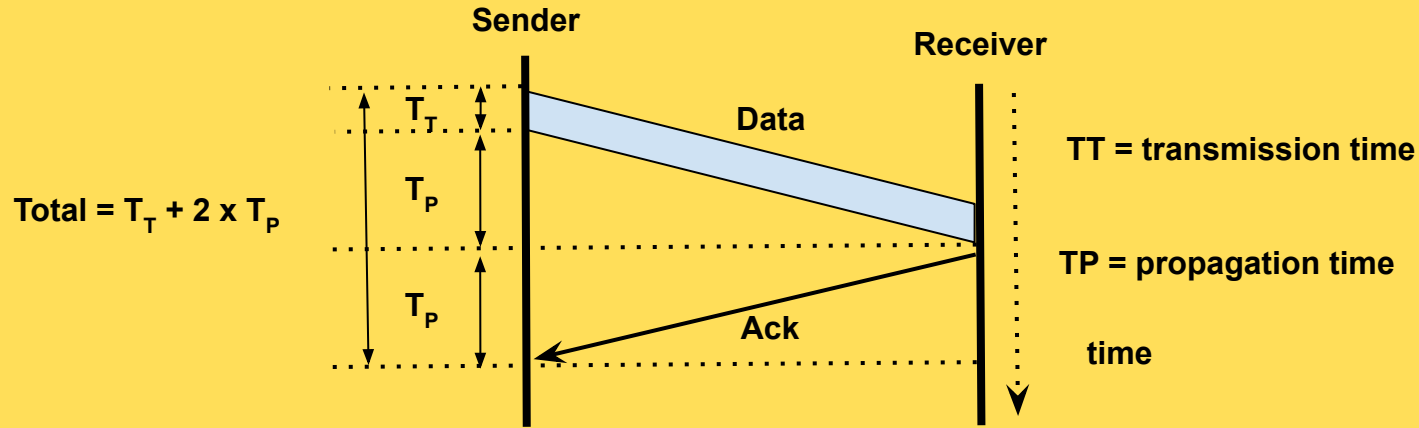
Propagation time = $10,000 / 200,000 = 50 \text{ ms}$, RTT = 100 ms

1000 bytes = 8000 bits = $8 \cdot 10^3$

Transmission time = $8 \cdot 10^3 / 10^6 = 8 \text{ ms}$

- How long does it take the sender to deliver each packet to the receiver?
- What is the difference for a protocol in which the sender was always transmitting?

Stop & Wait Protocol Performance



Each packet takes 8 ms to transmit, but in each 108 ms cycle only one packet is transmitted

1000 packets take 108 seconds to transfer instead of 8 seconds

Actual performance is $8/108 \approx 7.4\%$ of theoretical maximum

Channel Usage Rate

$$T_u = T_t / (T_t + 2 \times T_p)$$

or

$$T_u = T_t / (T_t + RTT)$$

T_u – usage rate

T_t – transmission time of a message,

T_p – end-to-end propagation time

RTT – round trip time ($2 \times T_p$)

Debit, RTT and Usage Rate

Usage rate of a channel using the stop & wait protocol in a transfer between two computers connected directly by a channel without errors, using 10,000-bit packets, varying the throughput and the RTT.

Usage Rate	Debit (Mbps)	T_T (ms)	RTT (ms)	Comments
99%	1	10	0,1	Low throughput, negligible RTT
50%	100	0,1	0,1	Reasonable throughput, negligible RTT
33%	1	10	20	Low throughput, low RTT
0,5%	100	0,1	20	Reasonable throughput, low RTT
4,8%	1	10	200	Low throughput, high RTT
0,05%	100	0,1	200	Reasonable throughput, high RTT
0,005%	1000	0,01	200	High throughput, high RTT

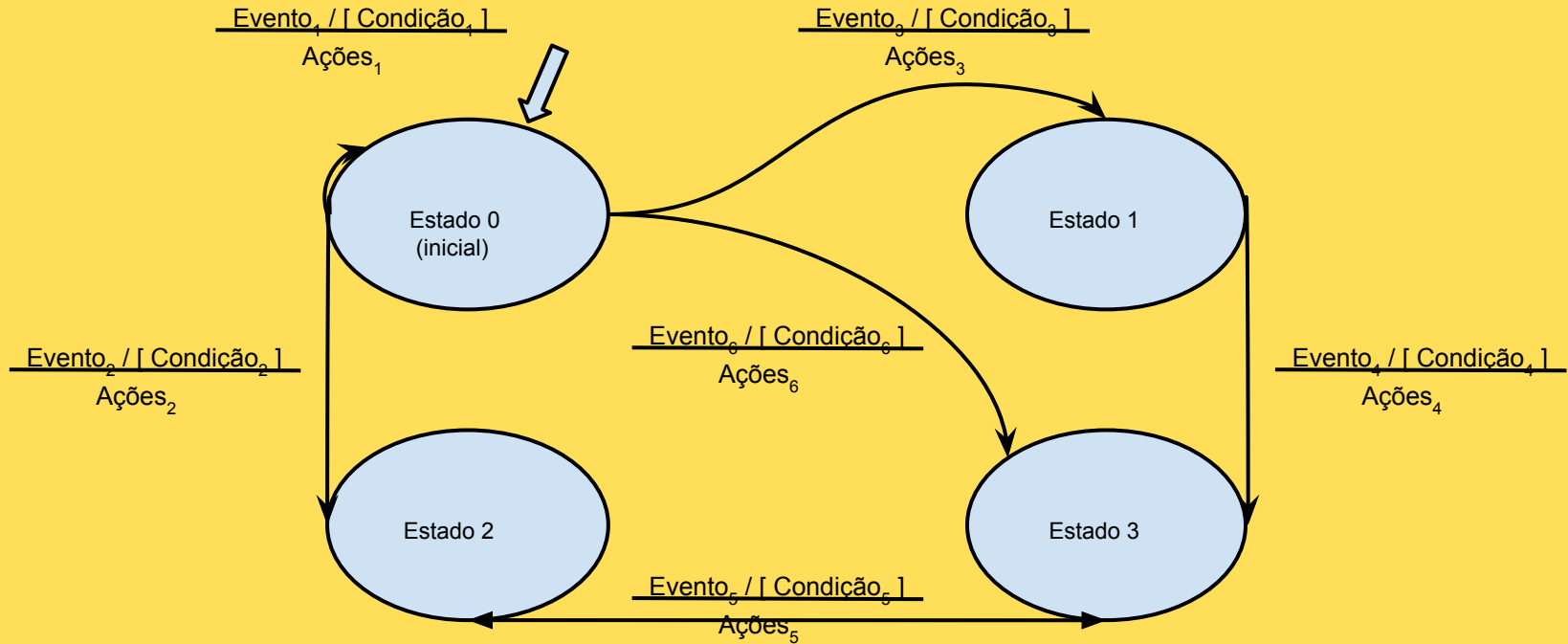
High Debit Channels

- When the channels have very high throughput, the transmission time decreases drastically. For example, if a channel has a capacity of 1 Gbps, transmitting 10,000 bits takes 10⁻⁵ seconds, i.e. 10 microseconds
- If the RTT is a few milliseconds, the channel utilization rate always tends towards very low values. In these cases, the average end-to-end useful throughput allowed by the protocol tends to:

$$\begin{aligned} &\text{Average end-to-end throughput of the S\&W protocol} \\ &= \\ &\text{Packet size / RTT} \end{aligned}$$

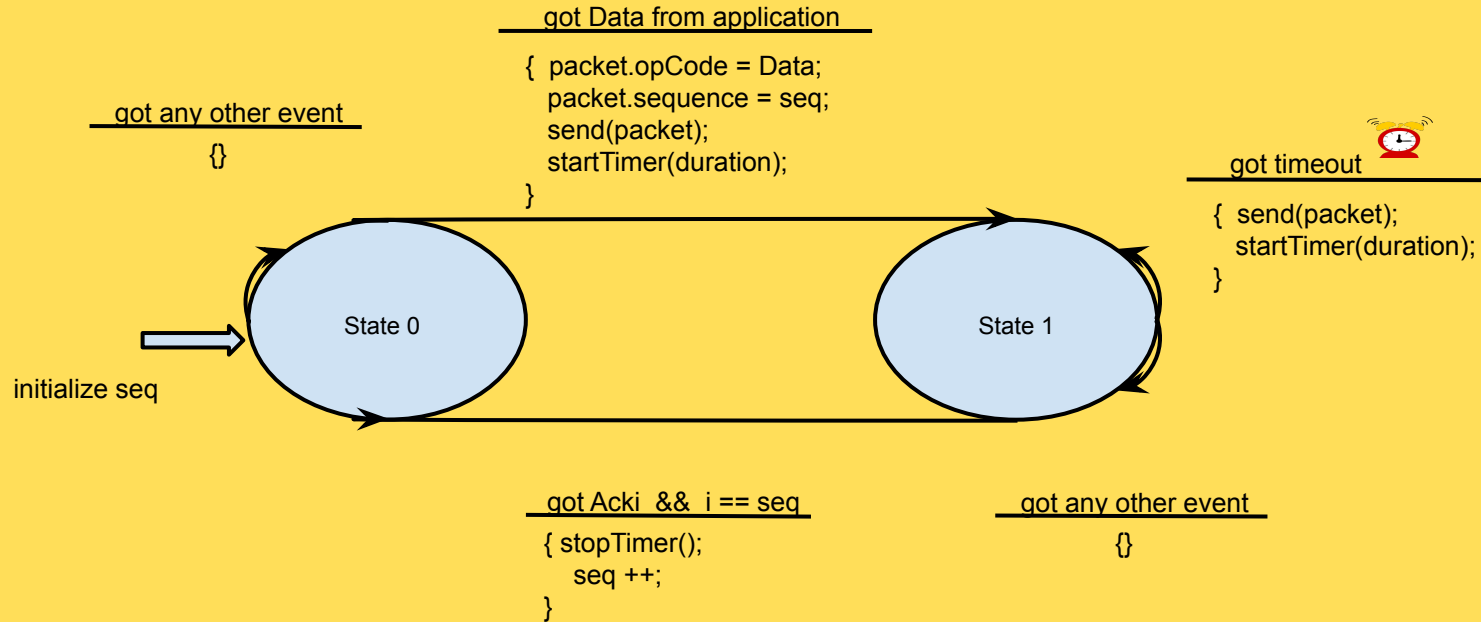
The end-to-end useful data output is called *goodput*

State Machines with Actions



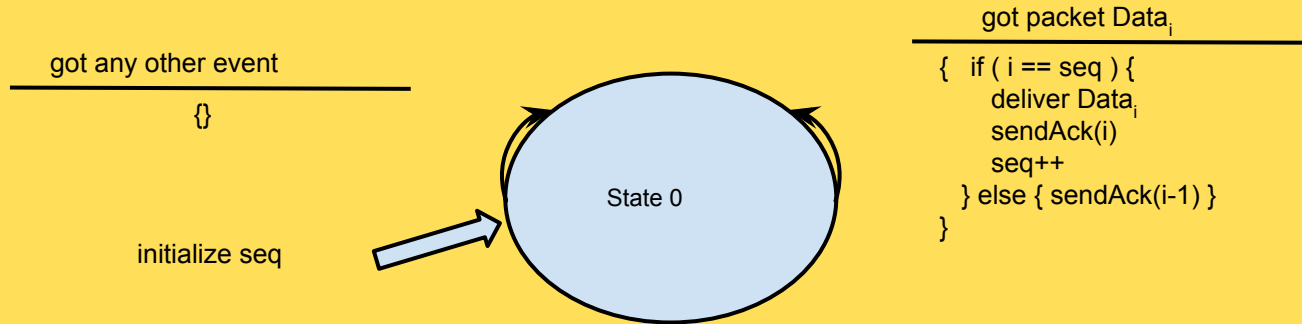
State Machines with Actions

Sender Stop & Wait



State Machines with Actions

Receiver Stop & Wait



Conclusions

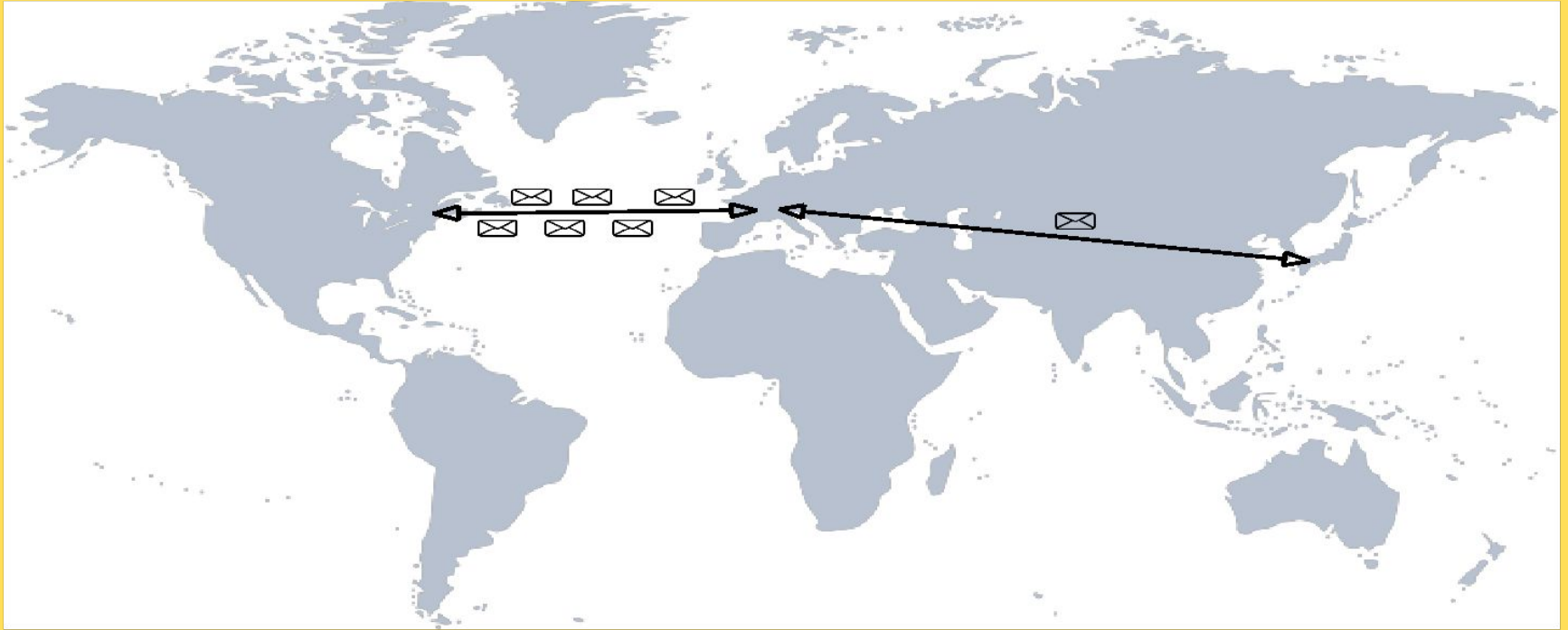
- Even if a channel or the Internet has errors and packet loss
 - It's possible to implement a reliable data transfer protocol at both ends
 - We just saw one, also known as stop & wait
- Unfortunately, it performs poorly when the end-to-end transit time is significant compared to the transmission time.
 - The sender advances at the rate allowed by the RTT, as
 - The receiver cannot send a new packet until the ACK of the last sent packet arrives.

Sliding Window Protocols

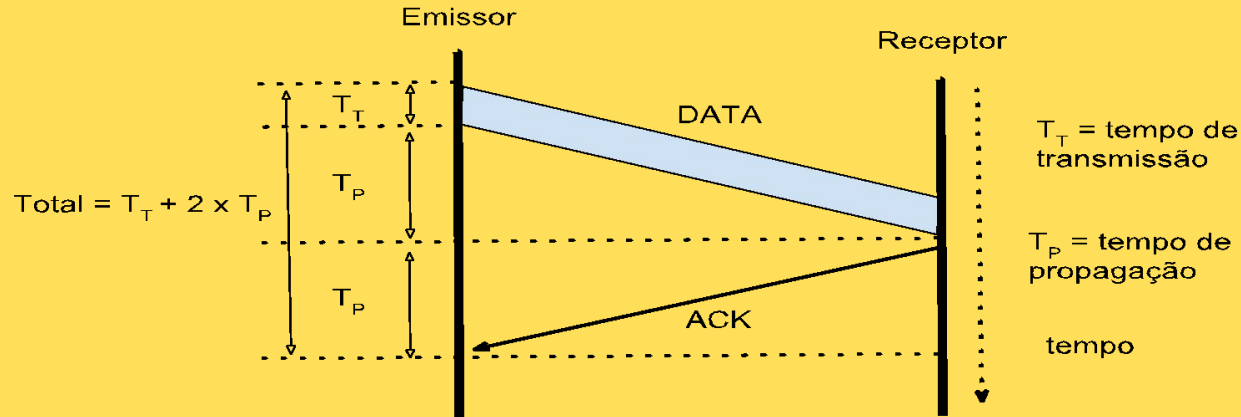
Improvement proposal

It is possible to improve the stop&wait protocol using a technique called sliding window or pipelining.

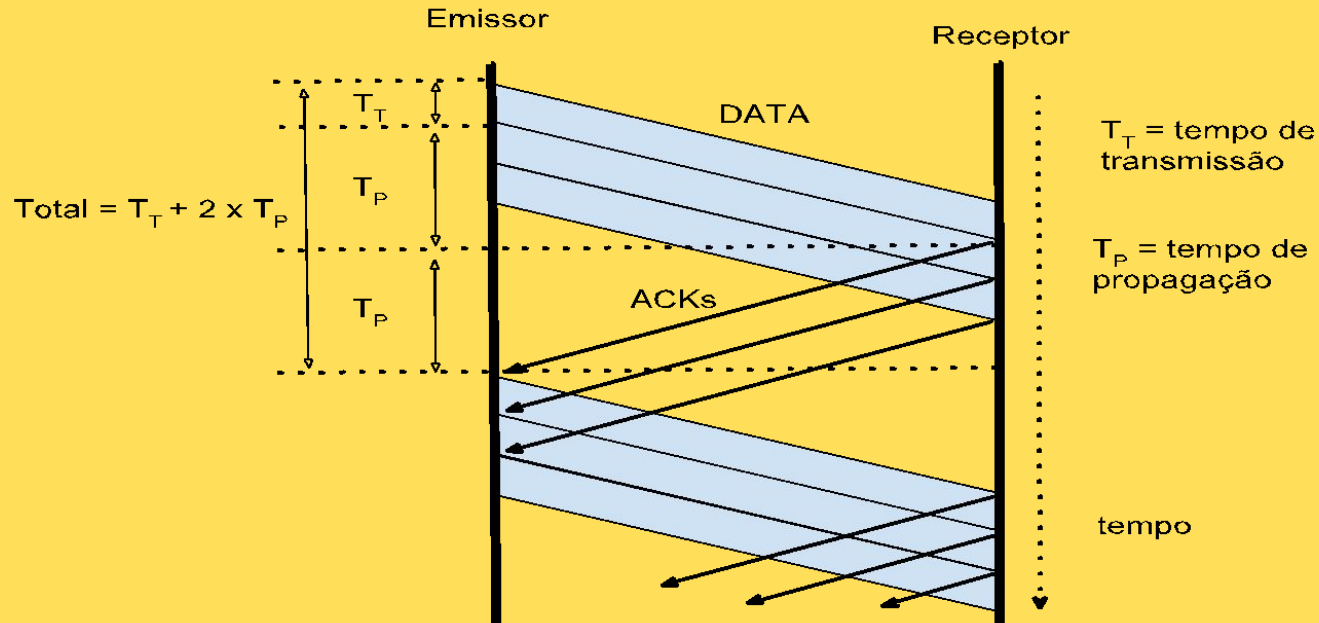
The problem



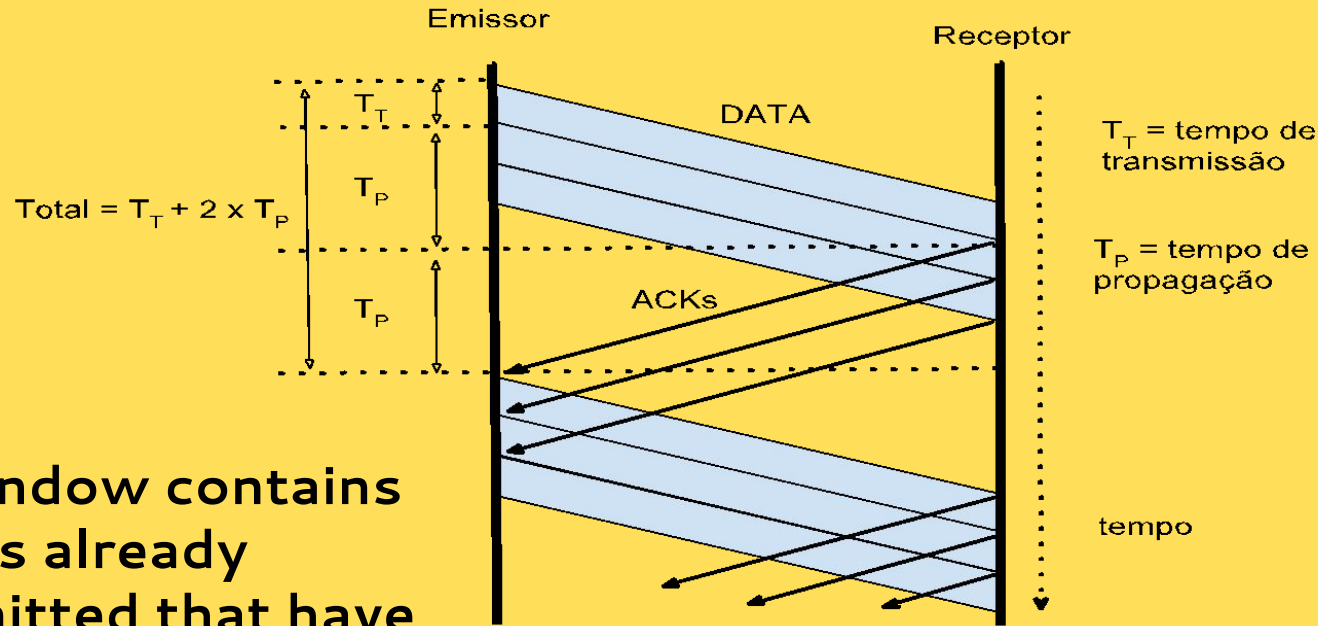
S&W Protocol Performance



Solution: Send "in Advance"

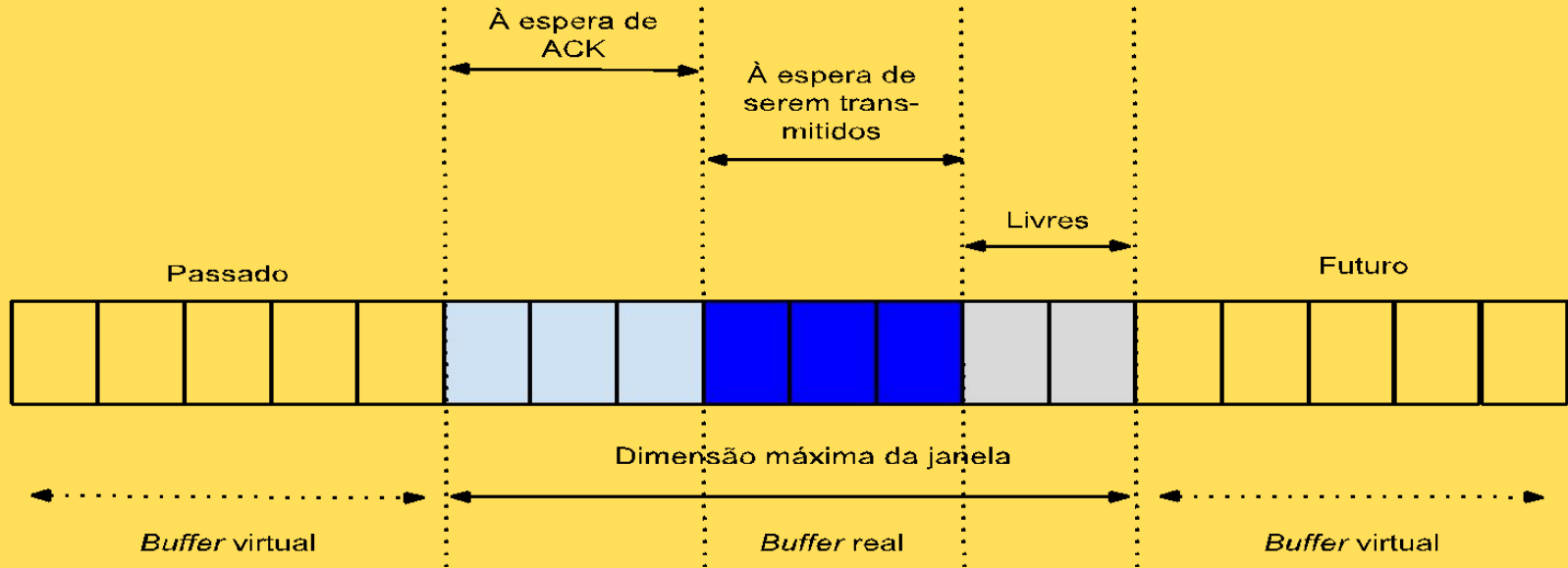


Sliding Window



The window contains packets already transmitted that have not yet been ACK'd. In the stop & wait protocol the window

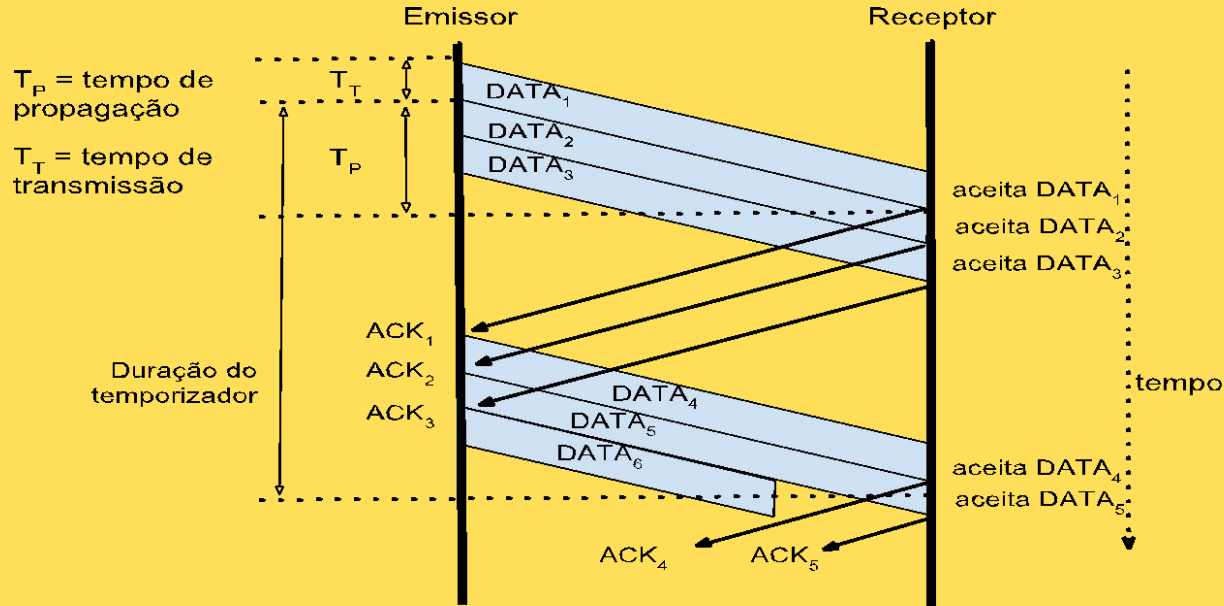
Window Operation



Window Operation

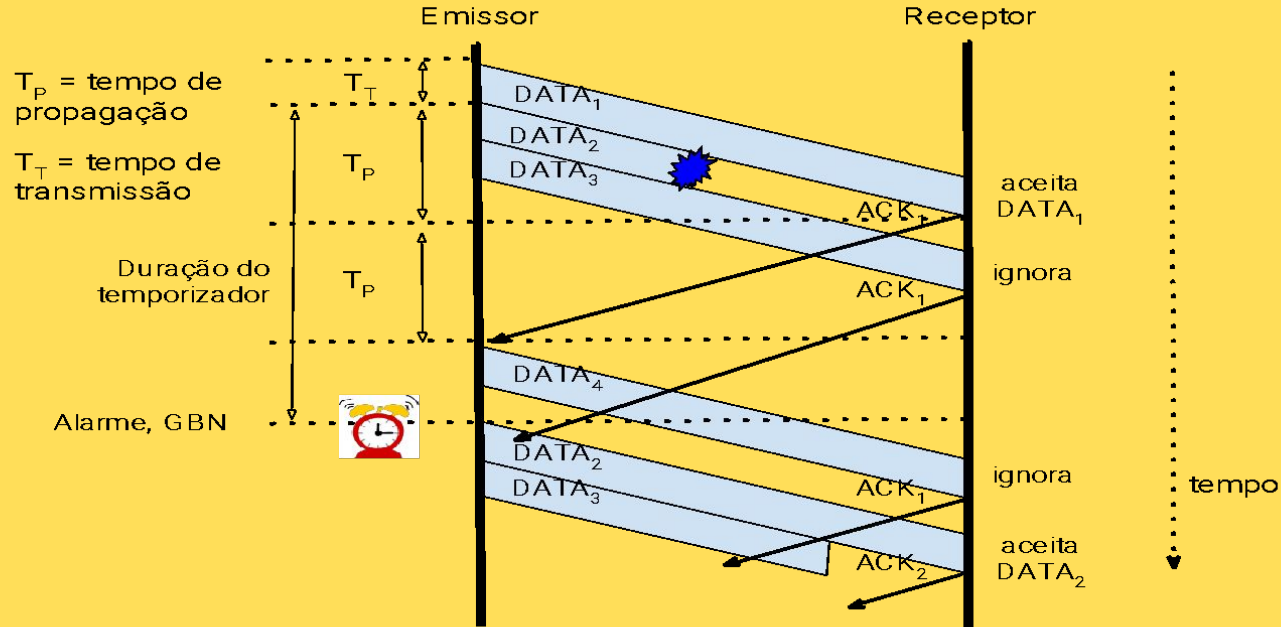
- The sender has a *buffer* (the window) where the packets that have already been transmitted and that have not yet been ACK'ed are, as well as the packets that are waiting to be transmitted.
- When ACKs arrive, the window slides to the right and already ACK'd packets can be forgotten as it is already certain that they were well received
- The size of the window is limited for the following reasons: flow control, possible waste in case of loss and network saturation control (see below)

One packet Window Receiver



A window for one packet at the receiver is sufficient if the application consumes incoming packets very quickly

But, with errors...



there is no room in the receiver for the out-of-order packets, so the sender has to go back to the beginning

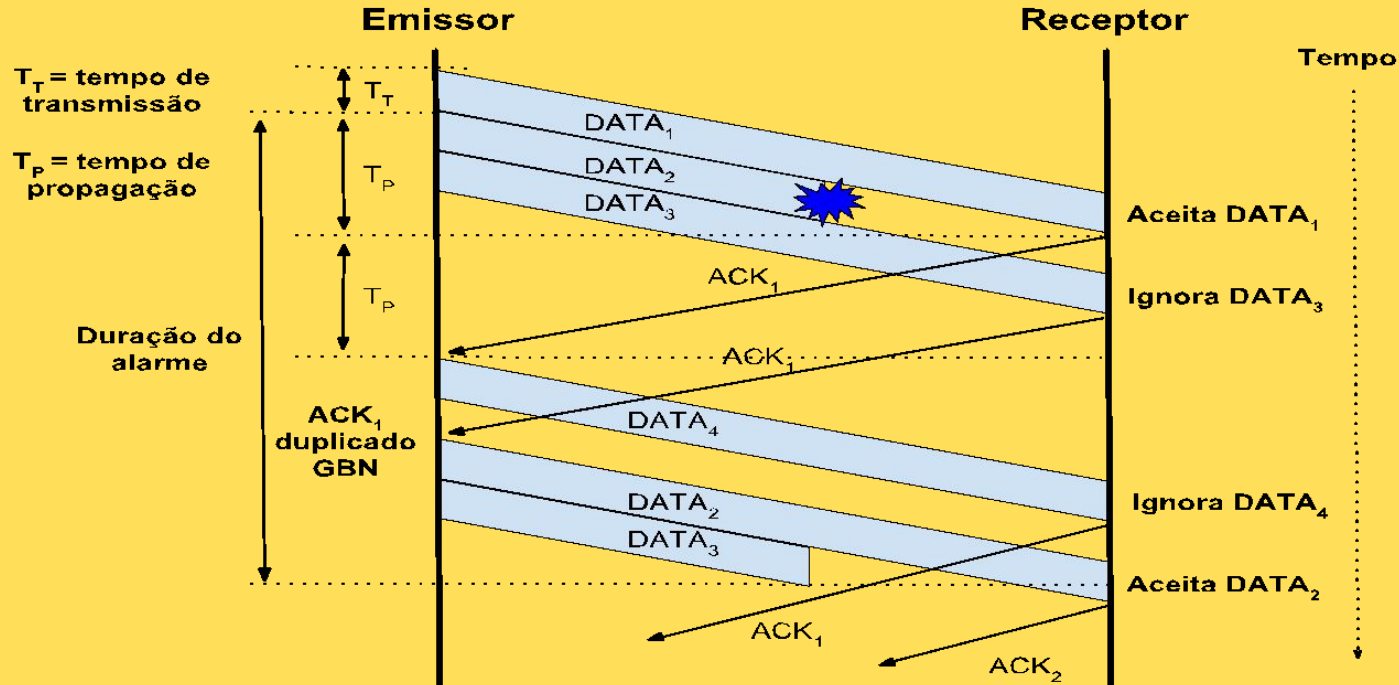
Go-Back-N (GBN)

- As the receiver does not keep the packets out of order, in the event of an anomaly the sender does not receive the appropriate ACK, and it can only restart the transmission of all packets in order from the packet whose ACK was missing, i.e., the one closest to the left of the window.
- There is always an alarm associated with the oldest emitted packet, which will have to be updated whenever this packet is ACK'ed
- There are other ways to detect that a packet has been lost before the alarm triggers (see below)

GBN and Recovery

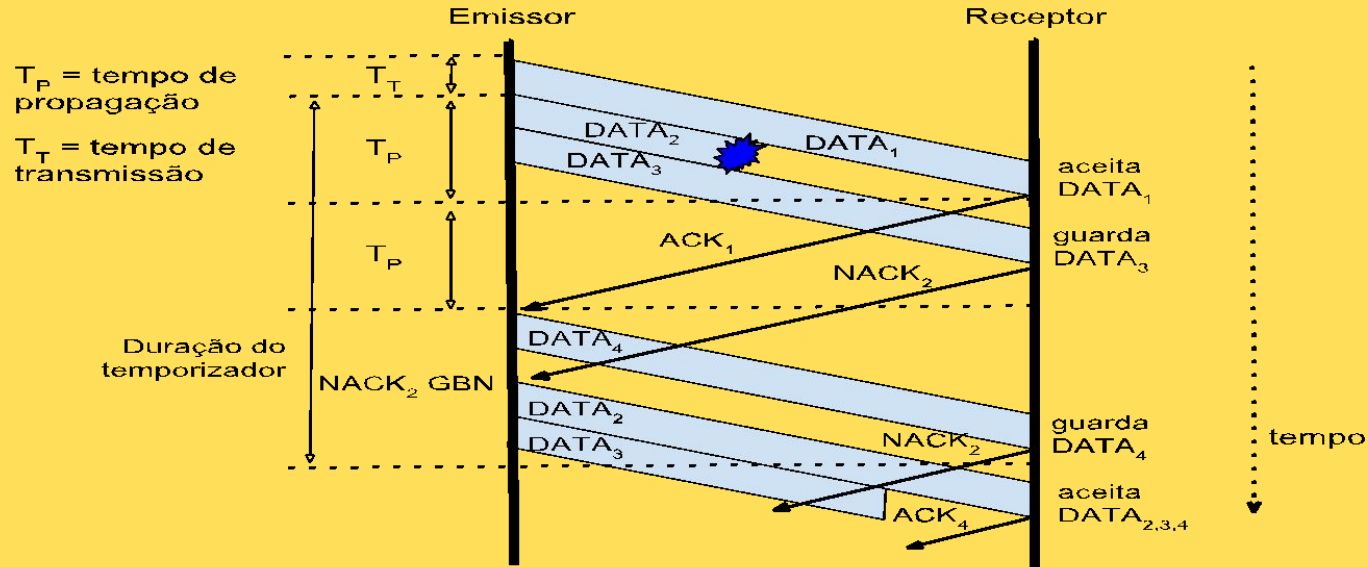
- The sooner the sender detects that a packet has been lost, the better, as this avoids continuing to send packets that will be discarded.
- There are at least two ways for the sender to detect that a packet is lost even if the alarm has not yet been triggered.
 - Interpreting duplicate ACKs as a sign that a packet has been lost
 - The receiver issues an explicit indication (a NACK) that a packet has been lost

GBN With Faster Recovery



A repeated cumulative ACK can be interpreted as a signal that a packet is lost and serves to enter GBN sooner

Receiver Window Larger Than a Packet

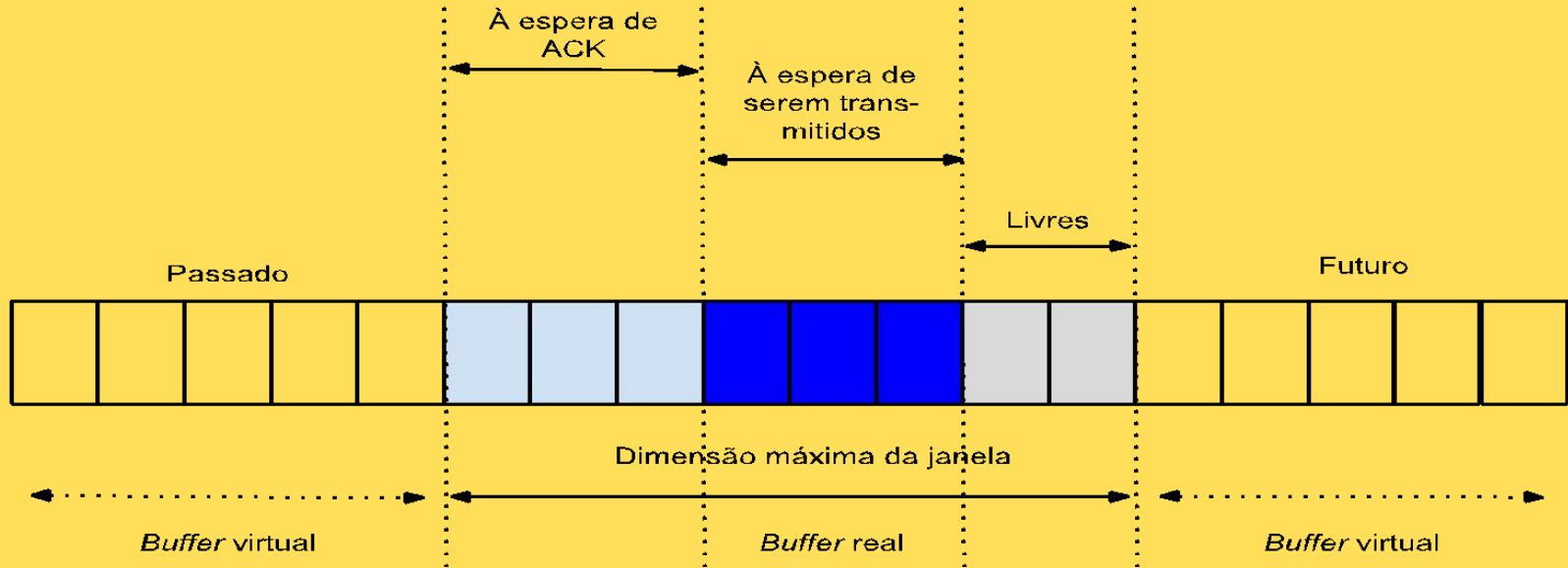


A receiver window greater than 1 also helps

Actions Performed by GBN

- For each message n received, the receiver
 - Keep the packet if it is in order
 - If it is out of order but has reception window > 1 try to save it too
 - In any case, it sends a cumulative ACK of everything it has received so far in the order, and optionally it can (also) send a NACK
- The sender sets an alarm (timeout) associated with the oldest packet sent
 - Whenever the timeout triggers on the sender, or it receives a NACK, it goes back and starts sending packets in the window again.
 - When the sender receives a “useful” ACK, i.e. to the left of the window, it advances the window and readjusts the alarm value

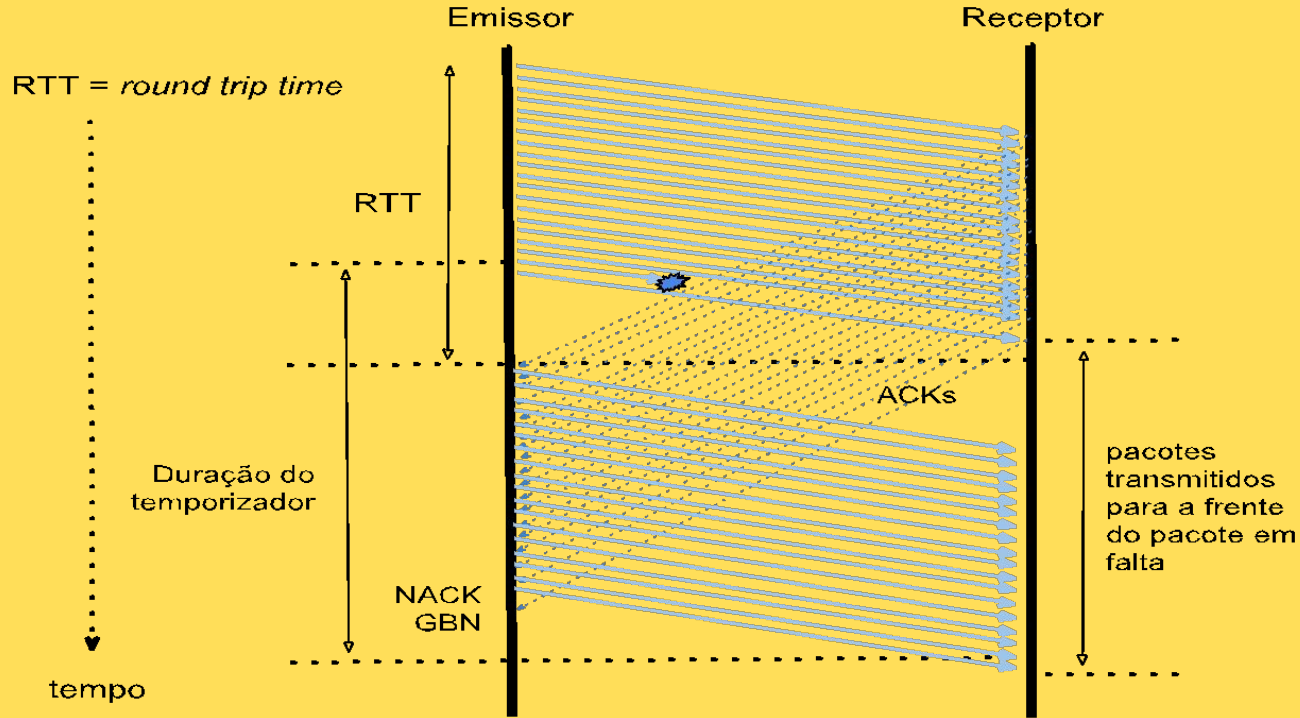
Sender Window



Can We Improve?

- When the RTT is very low, or the T_t / RTT value approaches 1, a relatively small emission window is enough to keep the emitter almost always emitting
- If there are transmission errors (admittedly not frequent) the GBN operation introduces delays and eventually duplicate transmissions. The penalty, if any, is proportional to the window size
- If the T_t value is too small (high capacity channel) and the RTT is too large (too long channel), the sender window has to be bigger and the GBN operation is very penalizing if there are errors, even if spaced apart.

GBN Recovery Cost



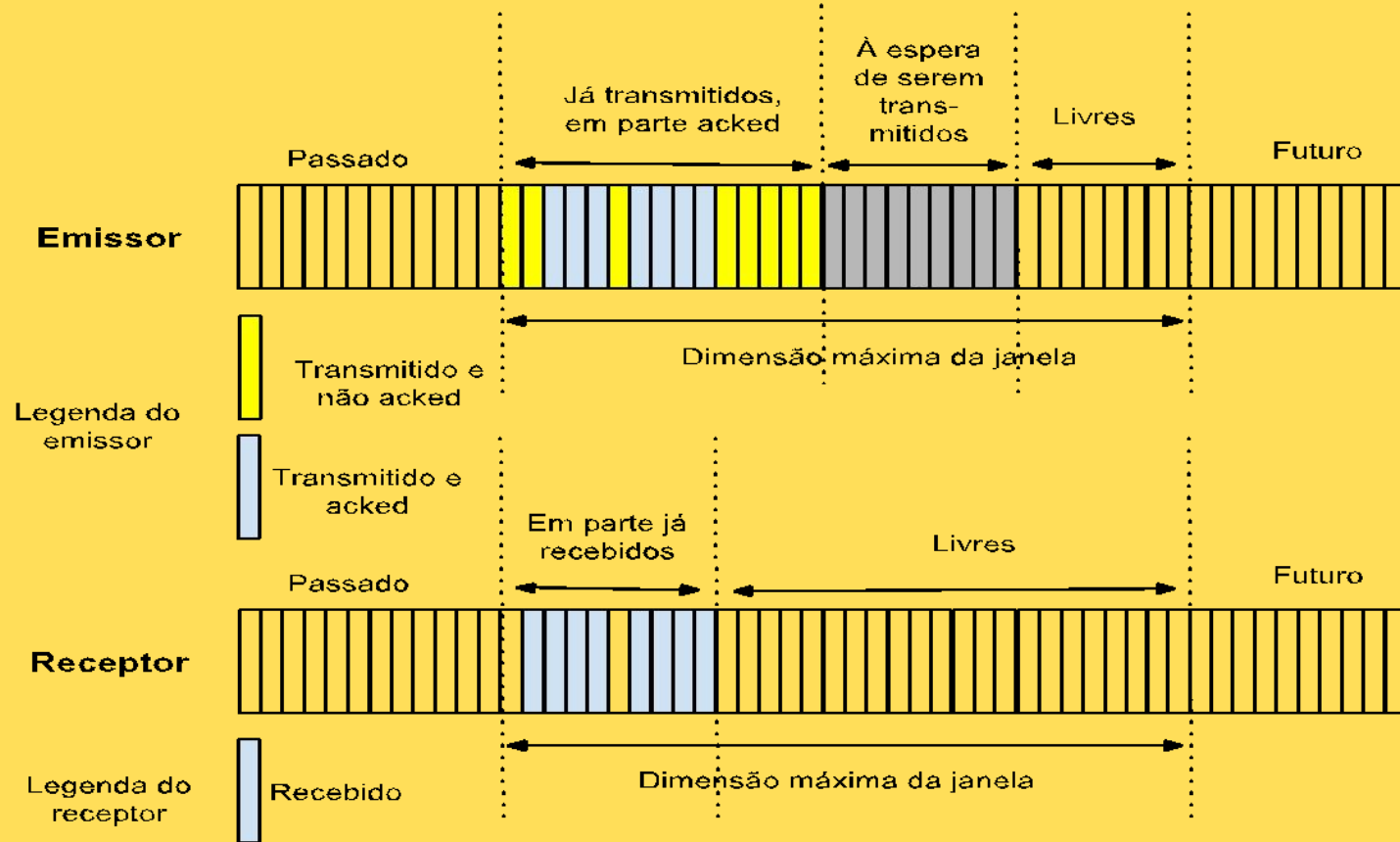
Selective Repeat (SR)

- It is possible to improve the algorithm by introducing independent ACKs for each sent and received packet.
- This means that only packets for which the sender has received a NACK or an alarm has been triggered are retransmitted.
- In these cases, the lost packet is retransmitted, but the GBN procedure is not performed. The sender always continues to send new messages as long as the window size allows
- This version of the protocol is called selective repeat (SR)

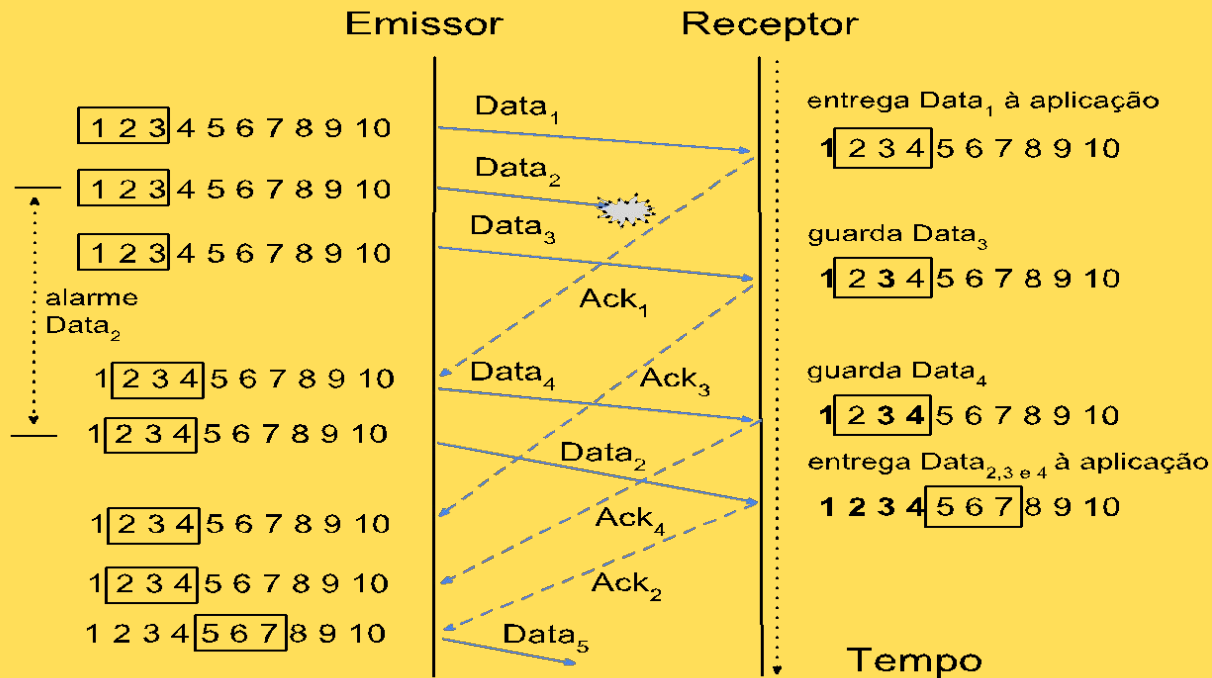
Functioning of the SR Protocol

- For each packet n received, the receiver sends the respective $ACK(n)$ (it can also send a NACK of the oldest missing packet if it has a hole)
- For each packet n sent, the sender sets a *timeout* $T(n)$
- Whenever a timeout $T(n)$ is triggered on the sender (or it receives a $NACK(n)$), packet n is resent
- The sender is still limited by the maximum size of its window

windows in SR



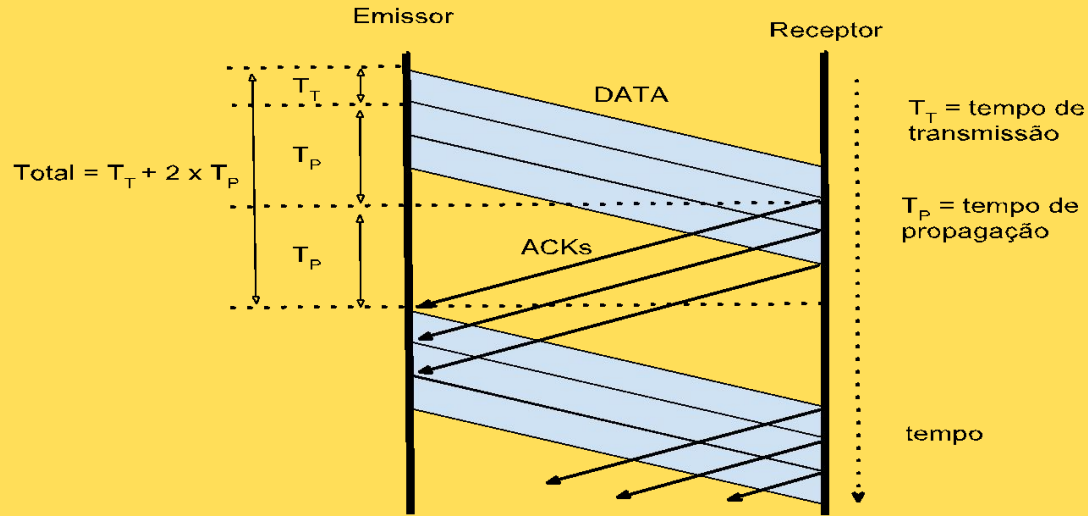
Example



Nomenclature

- Window (transmitter, receiver)
- $(1,1)$ – **stop & wait**
- $(N,*)$ – **sliding window** or pipelining
- $(N,1)$ – sliding window with **Go-Back-N** (GBN)
- (N,M) – with ACKs only for correctly received contiguous data (cumulative ACKs) – **still GBN** but with faster recovery
- (N,M) – with ACKs for well received data even if non-contiguous (selective ACKs) – **selective repeat**

Error Free Performance



Average end-to-end useful throughput of the protocol
 $\approx$ "Useful" size of the window in bits / $(T_T + \text{RTT})$
 $\approx N \times$ Throughput of the Stop & Wait protocol

Channels with Very High Debit

- When the channels have very high throughput, the transmission time decreases drastically. For example, if a channel has a capacity of 1 Gbps, transmitting 10,000 bits takes 10^{-5} seconds, i.e. 10 microseconds
- If the RTT is a few milliseconds and the T_T negligible, the average end-to-end throughput allowed by the protocol tends to:

Average end-to-end useful throughput of the protocol
= "Useful" window size in bits / RTT

- The end-to-end useful output is called goodput in English

Sender Window Dimension

- It doesn't pay to be much bigger than what you can continuously emit during an RTT
- It also depends on the protocol version as huge windows with GBN increase the potential waste when recovering from errors (are these frequent?)
- A window that's too large can also potentially *drown* a slow receiver (flow control) or saturate the network (saturation control)
- This is why TCP uses a window of variable size.

Receiver Window Dimension

- Theoretically, it doesn't pay to be greater than the sender's window.
- In the general case, it involves managing missing pieces before handing them over to the application, which is more complicated
- So the solution GBN and receiver window = 1 is not that bad as long as the T_T / RTT ratio is not too small and the packet loss rate is low
- TCP uses a constant size receiver window and may or may not use the SR version.

Alarm value

- Necessarily higher than the RTT
- If there are no buffers in the network between sender and receiver (e.g. both are connected by a direct point-to-point channel), the RTT is constant and the timeout value is easier to estimate
- If there are buffers in between (e.g. packet switches operating in store & forward mode and with significant queues), the RTT is variable and the timeout value is more difficult to estimate
- The TCP protocol uses a dynamically adjusted timeout value

Conclusions

- The relationship between transmission time and RTT is crucial for the performance of a stop & wait protocol
- When this throughput is low (e.g. $T_t \ll RTT$) it is essential to use sliding window protocols to improve throughput
 - It is the dominant case on the Internet when communication partners are "far away"
- It's a complex protocol with many adjustable parameters for each scenario.
 - The TCP protocol is a sliding window protocol that dynamically adapts these parameters to each concrete situation in which it is used.